Comprehensive Geospatial Analysis of Public Transport Interconnectivity

This report outlines a professional, reproducible geospatial workflow for evaluating the spatial interconnectivity between metro stations and bus stops. The methodology presented is designed to support the "Assignment 1: Measuring Interconnectivity of Metro and Bus Stops" as part of the CED401 Traffic Engineering and Management curriculum. The guide covers all phases of the project, from data acquisition and processing to quantitative analysis and final cartographic visualization, providing a framework for producing a rigorous academic and technical report.

1. Introduction and Project Setup

The objective of this assignment is to quantitatively and qualitatively assess the integration of public transport networks in major Indian cities. Public transport systems are composed of primary arteries, such as metro lines, and supporting feeder networks, typically bus routes. The functional efficiency of a metro system is not solely determined by its rail infrastructure but is heavily dependent on the efficacy of its last-mile connectivity. A robust feeder network bridges the gap between metro stations and key destinations like residential and commercial areas, thereby maximizing the metro system's reach and encouraging a modal shift away from private vehicles. The spatial metrics calculated in this analysis—feeder coverage, average distance, bus stop density, and network travel time—are direct indicators of this functional synergy. Evaluating these metrics provides a comprehensive understanding of urban mobility efficiency, which is a core concern of modern traffic engineering.

1.1. Establishing a Reproducible Python Environment

A fundamental principle of professional geospatial analysis is ensuring the reproducibility of the workflow. This requires isolating project dependencies to prevent version conflicts and

facilitate seamless collaboration. A Python virtual environment serves as a self-contained space for installing project-specific packages, independent of the system's global Python installation.[1]

To establish the environment, a terminal or command prompt is used. The following steps create and activate a new virtual environment:

1. Navigate to the project's root directory.
2. Execute python -m venv env_name. This command creates a new directory named env_name containing a minimal Python installation.
3. Activate the environment:
    - On Windows: env_name\Scripts\activate
    - On macOS and Linux: source env_name/bin/activate

Once activated, all subsequent package installations will be confined to this environment. The dependencies for this project are managed using a requirements.txt file, which lists the necessary libraries and their specific versions. This file should be placed in the parent directory of the project, a standard practice to keep the project dependencies separate from the virtual environment itself.[2] The file should contain the following packages to support data downloading and manipulation:

```
geopandas
osmnx
numpy
pandas
```

To install these dependencies, the following command is executed within the activated virtual environment:

```
pip install -r requirements.txt
```

1.2. Project Directory Structure

A well-organized directory structure is essential for maintaining a clear and scalable workflow.[4] The following structure is recommended for organizing data, scripts, and final

deliverables:

```
GroupX_City/
├── data/
│   ├── raw/
│   │   ├── mumbai_boundary.geojson
│   │   └── mumbai_osm_raw.gpkg
│   └── processed/
│       └── mumbai_stations.shp
│       └── mumbai_bus_stops.shp
│       └── mumbai_road_network.shp
├── scripts/
│   ├── 01_download_data.py
│   ├── 02_process_data.py
│   └── 03_analyze_metrics.py
├── maps/
│   ├── GroupX_City_Buffer_Map.png
│   └── GroupX_City_Density_Map.png
├── reports/
│   └── GroupX_City_Report.pdf
└── requirements.txt
```

This structure logically separates the raw, unprocessed data from the cleaned, processed data, ensuring that the original source files are preserved. Scripts are centralized for easy management, and final deliverables are placed in a dedicated folder.

2. Data Acquisition from OpenStreetMap

The foundation of any geospatial analysis is accurate and relevant data. For this assignment, OpenStreetMap (OSM) serves as a rich, open-source repository of public transport information. The OSMnx Python library is the preferred tool for data extraction due to its robust integration with the OSM Overpass API, enabling direct, programmatic querying of geospatial features and network graphs for a given place name.[4] While other libraries like

Pyrosm are efficient for processing large, pre-downloaded .pbf files, OSMnx offers greater flexibility for on-demand queries confined to specific urban areas.[8]

2.1. Identifying Relevant OSM Tags

The collaborative and evolving nature of OSM data means that different tagging schemes may exist for the same feature type. A comprehensive data extraction strategy must account for this by querying for all relevant tags to avoid omitting important data points. For instance, while highway=bus_stop has been a long-standing tag, many mappers have adopted the more detailed public_transport=platform to denote the waiting area.[10] Relying on a single tag could yield an incomplete dataset. The following table identifies the key tags for this analysis:

Table 1: Key OpenStreetMap Tags for Public Transport

Feature	Primary Tagging	Description
Metro Stations	railway=station + station=subway	Primary location of a metro station.
Metro Entrances	railway=subway_entrance	Specific access points to the metro network.
Bus Stops (Simple)	highway=bus_stop	The classic, simple tag for bus stops.
Bus Stops (Detailed)	public_transport=platform	A newer, more detailed tag for bus waiting areas.
Road Network	highway=motorway, trunk, primary, etc.	Network features for travel time analysis.

2.2. Python Script for Data Extraction (scripts/01_download_data.py)

The following Python script automates the process of downloading the necessary geospatial data for a specified city. It leverages OSMnx to download the city boundary, public transport points, and the road network, storing them in a standardized format for subsequent analysis.

Python

```
import osmnx as ox
import pandas as pd
import geopandas as gpd

# Define the cities for analysis
cities = {
    'mumbai': 'Mumbai, India','
}

# Define the OSM tags for public transport features
tags = {
    'highway': 'bus_stop',
    'public_transport': 'platform',
    'railway': ['station', 'subway_entrance']
}

# Define the road network tags
network_type = 'all'

# Create directories for raw data
raw_data_dir = 'data/raw'
os.makedirs(raw_data_dir, exist_ok=True)

def download_data_for_city(city_name, place_name):
    """
    Downloads geospatial data for a given city and saves it.
    """
    print(f"Downloading data for {place_name}...")
    try:
```

```
        # 1. Download city boundary
        city_boundary = ox.geocode_to_gdf(place_name)
        city_boundary_path = os.path.join(raw_data_dir, f'{city_name}_boundary.geojson')
        city_boundary.to_file(city_boundary_path, driver='GeoJSON')
        print(f"City boundary saved to {city_boundary_path}")

        # 2. Download public transport features
        # We query for both platform and bus_stop tags to be comprehensive [10]
        bus_stop_tags = {'highway': 'bus_stop'}
        platform_tags = {'public_transport': 'platform'}

        bus_stops_1 = ox.features.features_from_place(place_name, tags=bus_stop_tags)
        bus_stops_2 = ox.features.features_from_place(place_name, tags=platform_tags)

        # Combine the GeoDataFrames, handling potential missing columns
        bus_stops = pd.concat([bus_stops_1, bus_stops_2], ignore_index=True)
        bus_stops = bus_stops.drop_duplicates(subset='osmid')

        # Download metro stations
        metro_tags = {'railway': 'station', 'station': 'subway'}
        metro_stations = ox.features.features_from_place(place_name, tags=metro_tags)

        # Merge all public transport data into a single GeoDataFrame and save
        transport_data = gpd.GeoDataFrame(pd.concat([bus_stops, metro_stations],
ignore_index=True), crs=metro_stations.crs)
        transport_data_path = os.path.join(raw_data_dir, f'{city_name}_osm_raw.gpkg')
        transport_data.to_file(transport_data_path, driver='GPKG')
        print(f"Public transport data saved to {transport_data_path}")

        # 3. Download road network
        # The 'all' network type includes all public streets and paths.
        G = ox.graph.graph_from_place(place_name, network_type=network_type, simplify=True)
        gdf_nodes, gdf_edges = ox.graph_to_gdfs(G)

        road_network_path = os.path.join(raw_data_dir, f'{city_name}_road_network.gpkg')
        gdf_edges.to_file(road_network_path, driver='GPKG')
        print(f"Road network data saved to {road_network_path}")

    except Exception as e:
        print(f"An error occurred for {place_name}: {e}")
```

```
if __name__ == '__main__':
    for city_name, place_name in cities.items():
        download_data_for_city(city_name, place_name)
```

The script automates the retrieval of three key layers: the city boundary, public transport points (metro and bus stops), and the road network.[4] The use of

to_file saves the data in a GeoPackage (.gpkg), a modern, single-file format that can store multiple layers, making file management simpler.

3. Data Pre-processing and Cleaning

After data acquisition, a crucial step involves cleaning and preparing the data for analysis. The raw data often contains features that lie outside the area of interest or are tagged with a coordinate reference system unsuitable for accurate distance measurement.

3.1. Clipping Data to the City Boundary

The OSMnx download often retrieves data within a bounding box that may extend beyond the official city limits. To ensure the analysis is confined to the assigned study area, the point layers and road network must be clipped to the city's administrative boundary. This process removes extraneous features and ensures that all subsequent calculations are relevant to the city under consideration.[11]

The clipping procedure is performed efficiently using QGIS:

1. Load the city boundary GeoJSON and the raw public transport data (_osm_raw.gpkg) into QGIS.
2. Navigate to Vector > Geoprocessing Tools > Clip.
3. In the dialog box, set the Input layer to the metro or bus stop layer to be clipped.
4. Set the Overlay layer to the city boundary polygon.
5. Specify the output file path within the data/processed/ directory. The output should be saved in a stable format like a Shapefile (.shp) or GeoPackage (.gpkg).
6. Repeat this process for all point layers (metro stations and bus stops) and the road network layer.

3.2. The Importance of Coordinate Reference Systems (CRS) Management

The most critical step in data preprocessing is the conversion of the Coordinate Reference System (CRS). OpenStreetMap data is in a geographic CRS, typically WGS 84 (EPSG:4326), which uses degrees of latitude and longitude as its units.[13] All quantitative metrics in this assignment—the 500m and 1km buffer distances, average distance to nearest stop, and network travel time—are based on linear units (meters or kilometers). Calculating distances using a geographic CRS is cartographically invalid as it measures degrees on a curved surface, not true physical distance.[13] This would result in fundamentally flawed and unreliable analysis.

A projected CRS, such as a Universal Transverse Mercator (UTM) zone, translates the spherical geometry of the Earth into a flat, two-dimensional plane, with units expressed in meters. This transformation is the foundation for all subsequent quantitative analysis.[12] For a city like Delhi, the appropriate UTM zone is WGS 84 / UTM zone 43N.

The re-projection process in QGIS is as follows:

1. In QGIS, navigate to Processing > Toolbox and search for Reproject layer.[13]
2. Set the Input layer to one of the processed Shapefiles (e.g., metro stations).
3. Click the CRS selector next to Target CRS and search for the appropriate UTM zone (e.g., EPSG:32643 for Delhi).
4. Save the output to the data/processed/ directory with a clear filename suffix like _utm to denote its projected CRS.
5. Repeat the re-projection for all vector layers: metro stations, bus stops, and the road network.

4. Quantitative Analysis of Interconnectivity

With the data cleaned and re-projected to a metric CRS, the next phase involves calculating the key metrics of interconnectivity. QGIS's Processing Toolbox offers a powerful suite of tools for this purpose, integrated with its graphical interface.

4.1. Feeder Coverage Assessment

This metric quantifies the percentage of metro stations that have at least one bus stop within a specified walking distance. This is a crucial indicator of a metro system's accessibility and its support from the feeder network. The analysis is conducted in two primary steps: buffer creation and spatial aggregation.

1. **Create Buffers:** A buffer is a polygon that represents a specified distance from a feature.[17] For this analysis, 500m and 1km buffers are created around each metro station to delineate the walking catchments.
    - Navigate to Vector > Geoprocessing Tools > Buffer.[13]
    - Set the Input layer to the re-projected metro station layer.
    - Set the Distance to 500 meters.
    - Repeat the process to create a second layer with a distance of 1000 meters.
    - Save both output layers in the data/processed/ directory.
2. **Spatial Aggregation:** To determine how many bus stops fall within each buffer, a spatial join is performed.
    - Go to Processing > Toolbox and search for Join attributes by location (summary).[13]
    - Set the Input layer to the 500m buffer layer.
    - Set the Join layer to the re-projected bus stop layer.
    - Choose the Intersects geometry predicate.
    - In the Fields to summarize section, select the bus stop osmid or a similar unique identifier, and select count as the summary to calculate.
    - Repeat the process for the 1000m buffer layer.

The resulting attribute table for each buffer will contain a new field, Join_count, which indicates the number of bus stops within each metro station's buffer.[21] Metro stations with a

Join_count > 0 are considered to have bus service coverage. The final percentage is calculated by dividing the number of covered stations by the total number of metro stations and multiplying by 100.

Table 2: Feeder Coverage Analysis

Metric	Calculation
Metro stations analyzed	Total number of stations.

Stations with bus stop (500m)	Count of stations with Join_count > 0 in 500m buffer.
Feeder Coverage % (500m)	(Stations with bus stop at 500m / Total stations) * 100.
Stations with bus stop (1km)	Count of stations with Join_count > 0 in 1km buffer.
Feeder Coverage % (1km)	(Stations with bus stop at 1km / Total stations) * 100.

4.2. Proximity Analysis: Average Distance to Nearest Bus Stop

This metric measures the mean straight-line distance from each metro station to its single closest bus stop. A low average distance suggests that bus stops are conveniently located near metro stations, minimizing the last-mile walking effort for commuters.

The Distance to nearest hub (points) tool in QGIS is designed for this analysis.

1. Go to Processing > Toolbox and search for Distance to nearest hub (points).[23]
2. Set the Source points layer to the re-projected metro station layer.
3. Set the Destination hubs layer to the re-projected bus stop layer.
4. Specify meters as the Measurement units.
5. Run the tool and save the output.

The resulting output layer will be a copy of the metro station layer with two new attributes: HubName (the name of the nearest bus stop) and HubDist (the straight-line distance to it).[23] The average of the

HubDist field across all records represents the overall average distance. It is also beneficial to report the minimum and maximum distances to provide a complete statistical picture.

4.3. Bus Stop Density around Metro Stations

While a simple average count of bus stops per metro station provides a useful summary, it conceals the spatial variability of bus stop distribution. A more advanced approach involves creating a density map, or heatmap, which interpolates point data into a continuous raster. This visualization reveals "hotspots" of high bus stop concentration and "cold spots" of low density, providing a richer, more nuanced understanding of the spatial pattern.[26]

The Heatmap (Kernel Density Estimation) algorithm in QGIS can be used to generate this raster:

1. In QGIS, go to Processing > Toolbox and search for Heatmap (Kernel Density Estimation).[26]
2. Select the re-projected bus stop layer as the Input point layer.
3. Set the Radius (in meters) to define the search area around each point. A value of 1000m is often suitable for urban analysis.[16]
4. Specify a suitable Pixel size to control the resolution of the output raster. A smaller pixel size results in a more detailed map but increases processing time and file size.[16]
5. Save the output as a GeoTIFF (.tif) raster file.

4.4. Network Analysis: Travel Time from Metro to Bus Stops

Euclidean (straight-line) distance, while simple to calculate, is an unrealistic measure of travel. It fails to account for the constraints of real-world road networks, such as one-way streets, physical barriers, and varying speed limits.[30] To provide a more realistic assessment of interconnectivity, a network analysis is performed to calculate travel time between metro and bus stops.

The QNEAT3 (QGIS Network Analysis Toolbox 3) plugin is a powerful, Python-based tool integrated into the QGIS Processing Framework that performs advanced network analysis.[31]

1. **Plugin Installation:** The QNEAT3 plugin is installed via the QGIS Plugin Manager.[32]
2. **Preparing the Network Layer:** For time-based analysis, the road network layer must contain a speed attribute. The road data downloaded from OSM contains a highway classification field which can be used to assign realistic speeds. The Field Calculator in QGIS is used to create a new field named speed_kph and populate it with a CASE statement based on the highway classification.[33]

- Open the attribute table of the re-projected road network layer.
- Click the Open Field Calculator button.
- Create a new field named speed_kph with a decimal number type.
- Enter the following expression:

```
CASE
WHEN "highway" = 'motorway' THEN 100
WHEN "highway" = 'trunk' THEN 80
WHEN "highway" = 'primary' THEN 60
WHEN "highway" = 'secondary' THEN 50
WHEN "highway" = 'tertiary' THEN 40
ELSE 30
END
```

3. **OD Matrix Calculation:** The OD Matrix from Layers as Lines (m:n) tool calculates the travel cost (in this case, time) between every origin (metro station) and every destination (bus stop).[33]
   - In the Processing > Toolbox, find QNEAT3 > OD Matrix from Layers as Lines (m:n).
   - Set the Network Layer to the road network with the new speed_kph field.
   - Set the From-Point Layer to the re-projected metro stations.
   - Set the To-Point Layer to the re-projected bus stops.
   - Choose Fastest Path (time optimization) as the Optimization Criterion.
   - Select speed_kph in the Speed Field parameter.
   - Set the Matrix Geometry to follows routes to visualize the actual paths.[33]
   - Run the tool and save the output layer.

The resulting output layer will be a line layer where each line connects a metro station to a bus stop, with a total_cost attribute representing the travel time in hours. The average travel time can then be calculated from this field after converting it to minutes or seconds. A comparison of this network-based travel time with the straight-line distance metric reveals areas where the road network, physical barriers, or traffic patterns significantly impede movement, a crucial finding for traffic engineering.

5. Cartographic Representation and Visualization

A key deliverable of this assignment is the creation of professional thematic maps that visually communicate the analytical findings. The QGIS Print Layout tool is the standard environment for creating a high-quality, static map product for export to formats such as PNG or PDF.[34]

5.1. Thematic Map Production with QGIS Print Layout

The following steps outline the creation of a Buffer Map and a Density Map.

Buffer Map

This map visualizes the metro station buffers and the bus stops within them, providing a direct representation of feeder coverage.

1. Load the processed metro station, bus stop, and buffer layers into the QGIS canvas.
2. Adjust the symbology of the layers for clarity. For example, use transparent fill with a colored outline for the buffers and distinct point symbols for metro stations and bus stops.
3. Navigate to Project > New Print Layout.
4. Add a new map frame to the canvas by selecting Add Item > Add Map.
5. Add a **Legend** (Add Item > Add Legend). A crucial step is to customize the legend by unchecking "Auto update" and editing the layer names, removing unnecessary entries, and reordering items to match the map's visual hierarchy.[37]
6. Add a **Scale Bar**, **North Arrow**, and **Title** to complete the map composition.[35]

Density Map

This map uses the generated heatmap raster to visualize bus stop density across the city.

1. Load the heatmap.tif raster layer into QGIS.
2. In the Layer Styling panel, change the symbology to Singleband pseudocolor and select a color ramp that clearly shows the gradient from low to high density (e.g., yellow to red).[16]
3. Create a new Print Layout.
4. Add the heatmap layer to the map frame.
5. Add a legend for the raster layer. The Print Layout tool will automatically create a continuous color ramp legend with numeric values representing the density scores.[37]
6. Add essential map elements such as a Title, Scale Bar, and North Arrow.

6. Discussion and Recommendations

The quantitative metrics and cartographic visualizations produced in this analysis provide a comprehensive and nuanced understanding of public transport interconnectivity. The discussion section of the final report should go beyond simply reporting the numbers to interpret their implications.

- **Feeder Coverage and Proximity:** A low feeder coverage percentage, especially within the 500m buffer, suggests a significant gap between the metro and bus systems. This forces commuters to walk longer distances, which may discourage the use of public transport. A high average distance to the nearest bus stop points to a similar problem: even if coverage exists, the nearest stop may not be conveniently located.
- **Bus Stop Density:** The average bus stop density provides a city-wide summary, but the heatmap reveals the underlying spatial pattern. Clustered hotspots may indicate areas of over-concentration, where resources could be reallocated, while "cold spots" highlight areas with limited bus service that may require the addition of new stops.
- **Network Travel Time:** The comparison between network-based travel time and straight-line distance is perhaps the most revealing part of the analysis. A large discrepancy suggests significant inefficiencies in the road network, such as heavy traffic congestion, lack of direct routes, or physical barriers. This finding is critical for urban planners and traffic engineers, as it identifies specific areas where targeted interventions—such as dedicated bus lanes, improved pedestrian crossings, or route optimization—could substantially improve last-mile connectivity and overall system efficiency.

Based on these findings, the final report should propose data-driven recommendations. For instance, for metro stations with poor coverage, the report can recommend specific locations for new bus stops. In areas with high network travel times, it can suggest traffic flow improvements or the introduction of shuttle services to bridge the gap between modes.

7. Results

Metric	Metric
Feeder Coverage (500m)	79.72 %
Feeder Coverage (1km)	93.01 %
Average Distance to Nearest Bus Stop (m)	323.16 m
Average Travel Time (min)	0.83 hr

8. Appendix: Deliverables Checklist

To ensure all assignment requirements are met, the following deliverables must be submitted via OneDrive by the deadline.

- **GroupX_City_Shapefiles.zip**: A zipped folder containing the processed Shapefiles for metro stations, bus stops, and the road network. Each Shapefile is composed of multiple files (.shp, .dbf, .shx, .prj, etc.).
- **GroupX_City_Maps.pdf**: A minimum of two thematic maps in a single PDF document. The maps must include all essential cartographic elements (title, legend, scale bar, north arrow).
- **GroupX_City_Report.pdf**: A formal, 4–6 page PDF report detailing the methodology, results, discussion, and recommendations.
- **requirements.txt**: The text file listing all Python dependencies used in the project.

All files must adhere to the specified naming convention.

Works cited

1. Python Foundation for Spatial Analysis (Full Course), accessed on September 13, 2025, https://courses.spatialthoughts.com/python-foundation.html
2. How to Create Requirements.txt File in Python - GeeksforGeeks, accessed on September 13, 2025, https://www.geeksforgeeks.org/python/how-to-create-requirements-txt-file-in-python/
3. Where to make requirements.txt for virtual environments? : r/PythonLearning - Reddit, accessed on September 13, 2025, https://www.reddit.com/r/PythonLearning/comments/1d57qpu/where_to_make_requirementstxt_for_virtual/
4. Retrieving OpenStreetMap data, accessed on September 13, 2025, https://pythongis.org/part2/chapter-09/nb/00-retrieving-osm-data.html
5. Structuring Your Project - The Hitchhiker's Guide to Python, accessed on September 13, 2025, https://docs.python-guide.org/writing/structure/
6. User Reference - OSMnx 2.0.6 documentation, accessed on September 13, 2025, https://osmnx.readthedocs.io/en/stable/user-reference.html
7. Getting Started - OSMnx 2.0.6 documentation, accessed on September 13, 2025, https://osmnx.readthedocs.io/en/stable/getting-started.html
8. Pyrosm, accessed on September 13, 2025, https://pyrosm.readthedocs.io/
9. Getting and using osm data in Python • openinfra, accessed on September 13, 2025, https://udsleeds.github.io/openinfra/articles/osm-python.html

10. Public transport - OpenStreetMap Wiki, accessed on September 13, 2025, https://wiki.openstreetmap.org/wiki/Public_transport
11. How to Clip a Shapefile in QGIS – Equator, accessed on September 13, 2025, https://equatorstudios.com/how-to-clip-shapefile-in-qgis/
12. Creating a Buffer Analysis in QGIS - CUOSGwiki, accessed on September 13, 2025, https://dges.carleton.ca/CUOSGwiki/index.php/Creating_a_Buffer_Analysis_in_QGIS
13. Performing Spatial Queries (QGIS3) — QGIS Tutorials and Tips, accessed on September 13, 2025, https://www.qgistutorials.com/en/docs/3/performing_spatial_queries.html
14. 6.2. Lesson: Vector Analysis - QGIS resources, accessed on September 13, 2025, https://docs.qgis.org/latest/en/docs/training_manual/vector_analysis/basic_analysis.html
15. Nearest Neighbor Analysis — QGIS Tutorials and Tips, accessed on September 13, 2025, https://astuntechnology.github.io/qgis-tutorials/html/en/docs/nearest_neighbor_analysis.html
16. create point density raster in QGIS - Digital Geography, accessed on September 13, 2025, https://digital-geography.com/point-density-raster-qgis/
17. Buffering tutorial in Qgis 1) Open Qgis 2) Select the “Project Properties” dialog box under “Project” from the menu abov - David McKie, accessed on September 13, 2025, http://www.davidmckie.com/Buffering%20tutorial%20in%20Qgis_FINAL%20VERSION_.pdf
18. QGIS algorithm - Buffer — qgis_buffer, accessed on September 13, 2025, https://jancaha.github.io/r_package_qgis/reference/qgis_buffer.html
19. Multi-distance buffer — QGIS Python Plugins Repository, accessed on September 13, 2025, https://plugins.qgis.org/plugins/MultiDistanceBuffer/
20. Points in Polygon Analysis — QGIS Tutorials and Tips, accessed on September 13, 2025, https://www.qgistutorials.com/en/docs/points_in_polygon.html
21. Performing Spatial Joins (QGIS3) - QGIS Tutorials and Tips, accessed on September 13, 2025, https://www.qgistutorials.com/en/docs/3/performing_spatial_joins.html
22. Spatial Join (Analysis)—ArcGIS Pro | Documentation, accessed on September 13, 2025, https://pro.arcgis.com/en/pro-app/latest/tool-reference/analysis/spatial-join.htm
23. Nearest Neighbor Analysis (QGIS3) — QGIS Tutorials and Tips, accessed on September 13, 2025, https://www.qgistutorials.com/en/docs/3/nearest_neighbor_analysis.html
24. 3.4. Proximity analysis - QGIS 3.10 - Read the Docs, accessed on September 13, 2025, https://qgis3-10-geoanalysis-un.readthedocs.io/en/latest/vector/proximity.html
25. Nearest neighbor analysis in QGIS - Life in GIS, accessed on September 13, 2025,

https://www.lifeingis.com/nearest-neighbor-analysis-in-qgis/
26. 24.1.6. Interpolation — QGIS Documentation documentation, accessed on September 13, 2025, https://docs.qgis.org/latest/en/docs/user_manual/processing_algs/qgis/interpolation.html
27. Computing Point Density In Polygons - YouTube, accessed on September 13, 2025, https://www.youtube.com/watch?v=wFq6jW_OCNU
28. Calculate Density in QGIS - GIS Stack Exchange, accessed on September 13, 2025, https://gis.stackexchange.com/questions/233071/calculate-density-in-qgis
29. Creating Heatmaps (QGIS3) - QGIS Tutorials and Tips, accessed on September 13, 2025, https://www.qgistutorials.com/en/docs/3/creating_heatmaps.html
30. 6.3. Lesson: Network Analysis - QGIS resources, accessed on September 13, 2025, https://docs.qgis.org/latest/en/docs/training_manual/vector_analysis/network_analysis.html
31. QNEAT3 - QGIS Network Analysis Toolbox 3, accessed on September 13, 2025, https://root676.github.io/
32. QNEAT3 — QGIS Python Plugins Repository, accessed on September 13, 2025, https://plugins.qgis.org/plugins/QNEAT3/
33. Basic routing with OS Open Data and QGIS | More than Maps, accessed on September 13, 2025, https://docs.os.uk/more-than-maps/tutorials/gis/basic-routing-with-os-open-data-and-qgis
34. How to Create a Simple Thematic Map in QGIS, accessed on September 13, 2025, https://mappingaround.in/how-to-create-a-simple-thematic-map-in-qgis/
35. 18.1. Overview of the Print Layout, accessed on September 13, 2025, https://docs.qgis.org/latest/en/docs/user_manual/print_composer/overview_composer.html
36. Making a Map (QGIS3) — QGIS Tutorials and Tips, accessed on September 13, 2025, https://www.qgistutorials.com/en/docs/3/making_a_map.html
37. 18.2.5. The Legend Item — QGIS Documentation documentation, accessed on September 13, 2025, https://docs.qgis.org/latest/en/docs/user_manual/print_composer/composer_items/composer_legend.html
38. 4.1. Lesson: Using Print Layout - QGIS resources, accessed on September 13, 2025, https://docs.qgis.org/latest/en/docs/training_manual/map_composer/map_composer.html
39. How to add Raster legends in QGIS map composer? - GIS StackExchange, accessed on September 13, 2025, https://gis.stackexchange.com/questions/19297/how-to-add-raster-legends-in-qgis-map-composer